

6 图遍历
Graph Traversal

- 图的遍历是求解图问题的基础。
- 和树的遍历类似，图的遍历希望从图中某一顶点出发，对其余各个顶点都访问一次，但比树的遍历要复杂得多。
- 图的任一顶点都有可能和其余顶点相邻接，因此在访问了某顶点后，可能沿着某条路径搜索以后，又回到该顶点。
- 通常有两种遍历图的方法：深度优先搜索、广度(宽度)优先搜索。他们都适合于无向图和有向图。
- 本章重点内容是介绍两种图遍历算法，并学习图遍历算法的一些应用。

图的两种遍历方法

- 深度优先搜索(Depth-First Search, DFS)
- 宽度优先搜索(Breadth-First Search, BFS)

深度优先搜索(Depth-First Search, DFS)

给定有向或是无向图G＝(V,E)，DFS工作过程如下：

1. 将所有的顶点标记为”unvisited”。
2. 选择一个起始顶点,不妨称为v ∈V,并将之标记为”visited”。
3. 选择与v相邻的任一顶点，不妨称之为w，将w标记为”visited”。
4. 继续选择一个与w相邻且未被访问的顶点作为x；将x标记为”visited”。继续选择与x相邻且未被访问的顶点。
5. 此过程一直进行，直到发现一个顶点y，邻接于y的所有顶点都已经被标记为”visited”。此时，返回到最近访问的顶点，不妨称之为z，然后访问和z相邻且标记为”unvisited”的顶点。
6. 上述过程一直进行，直到返回到起始顶点v。

- 深度优先搜索生成树(depth-first search spanning tree)
- 深度优先搜索生成森林(depth-first search spanning forest)
- predfn：在图的深度优先搜索生成树(森林)中顶点的先序号。所谓先序号，是指按照先序方式访问该生成树，该顶点的序号。
- postdfn：在图的深度优先搜索生成树(森林)中顶点的后序号。所谓后序号，是指按照后序方式访问该生成树，该顶点的序号。
- 对边(v,w)进行探测的含义是：在调用dfs(v)的过程中，检查(v,w)以测试w是否已经被访问过(”visited”)。

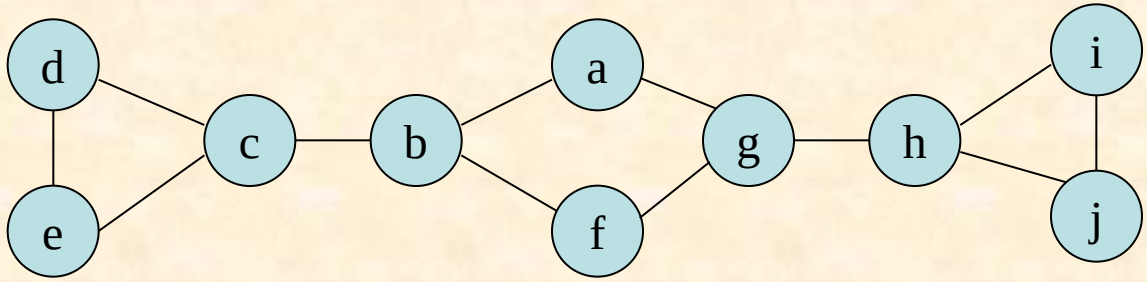
输入：无向图或有向图G= (V,E)
输出：深度优先搜索树(森林)中每个顶点的先序号、后序号

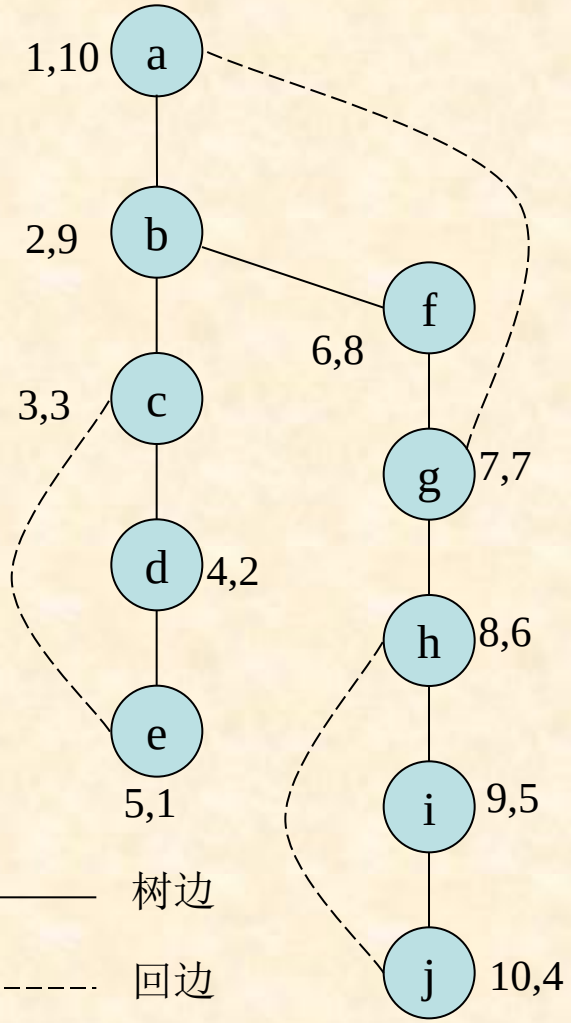
```
1. predfn←0; postdfn←0  //计数器，在使用DFS解决某些实际问题时用到
2. for v∈V
3.   visited[v] ← false
4. end for
5. for v∈V     //从一个顶点v出发，可能无法遍历全部顶点
6.   if visited[v] = false  then dfs(v)
7. end for

dfs(v)
1.     visited[v] ←true
2.     predfn←predfn+1
3.     for (v,w)∈E
4.      if visited[w]=false then dfs(w)
5.     end for
6.     postdfn←postdfn+1.
```

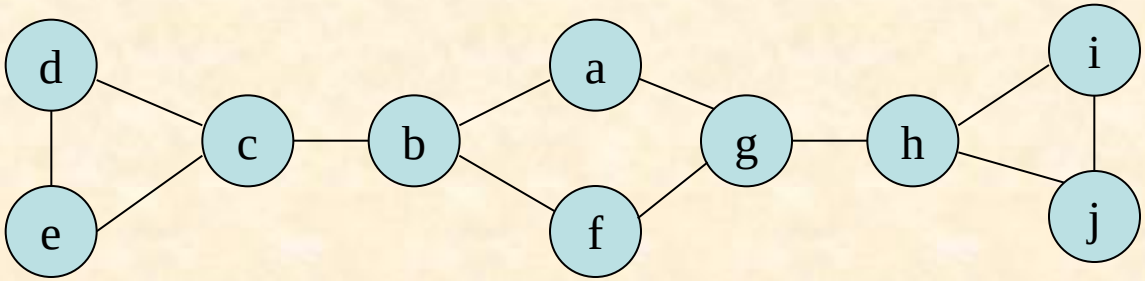
无向图情形

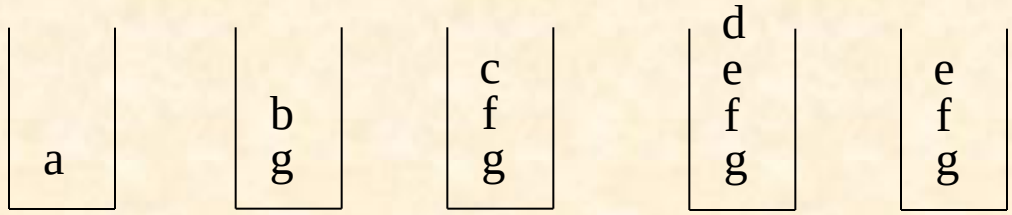
- 无向图G= (V,E)，深度优先遍历后，G中的边可以分为如下类型：
- 树边(Tree edges)— 深度优先搜索生成树中的边：探测边(v,w)时，w是“unvisited”状态，则边(v,w)是树边。
- 回边(Back edges)—G中除却树边的所有其它边。



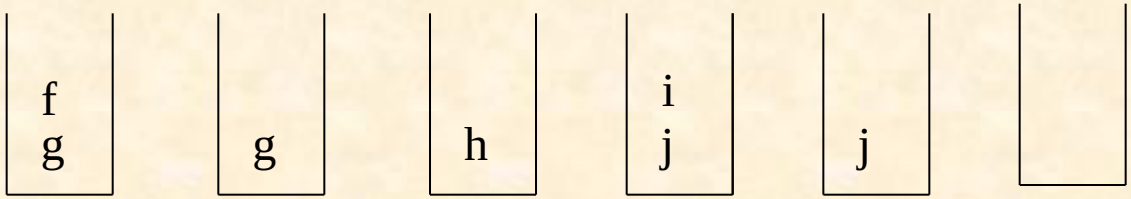


postdfn=1: 该顶点是第一个不能继续向深度前进的顶点；
或是称为第一个完成深度搜索的顶点。

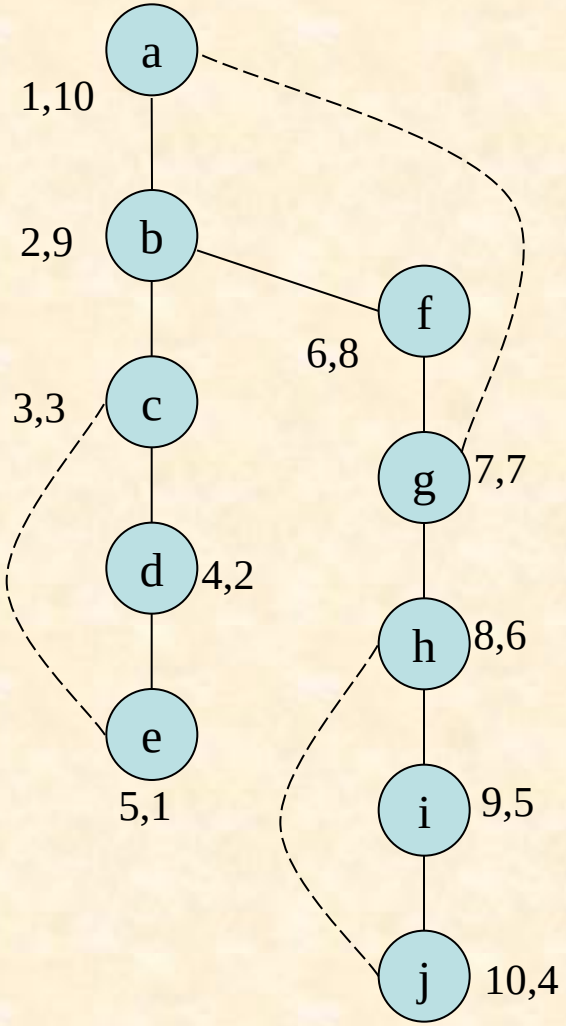




a b c d



e f g h i j



使用栈的方式来实现DFS

借助一个堆栈实现迭代形式的DFS

```
dfs(v)
1.    visited[v] ←true
2.    predfn←predfn+1
3.    for (v,w)∈E  //总共的循环次数
4.        if visited[w]=false then dfs(w)
5.    end for
6.    postdfn←postdfn+1.
```

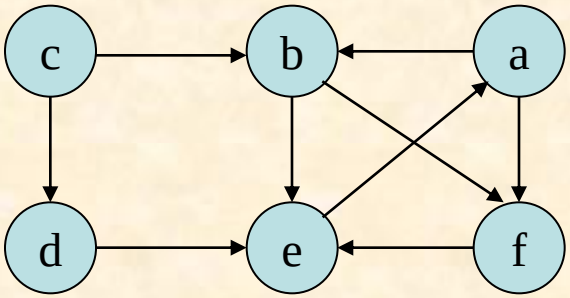
递规形式

```
dfs(v)
1.    Push(v,S)
2.    visited[v] ←true
3.    While S!={}
4.        v ←Pop(S)  //visit
5.        for (v,w)∈E
6.          if visited[w]=false then
7.                Push(w,S)
8.                visited[w] = true
9.           end if
10.       end for
11.   end while
```

迭代形式

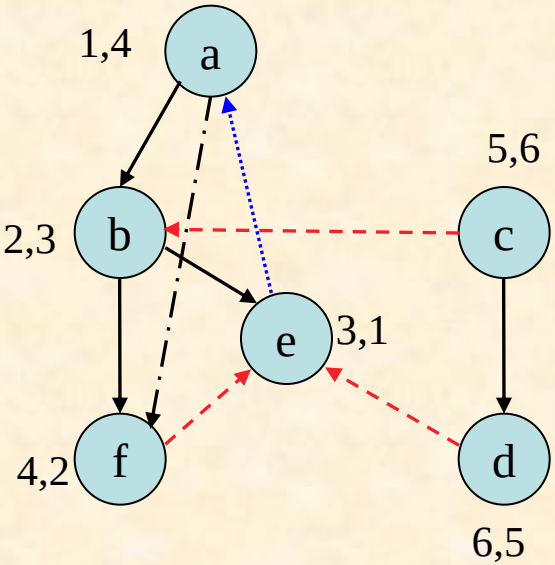
有向图情形

- 有向图G= (V,E)，深度优先遍历后，会生成一个或是多个(有向)深度优先搜索生成树。G中的边可以分为如下4种类型：
- 树边(Tree edges)－ 深度优先搜索生成树中的边：探测边(v,w)时，w是“unvisited”状态，则边(v,w)是树边。
- 回边(Back edges)－在迄今为止所构建的深度优先搜索生成树中，w是v的祖先，并且在探测(v,w)时，w已经被标记为”visited”,则(v,w)为回边。
- 前向边(Forward edges)－在迄今为止所构建的深度优先搜索生成树中，w是v的后裔，并且在探测(v,w)时，w已经被标记为”visited”,则(v,w)为前向边。
- 横跨边(Cross edges)－所有其他的边。

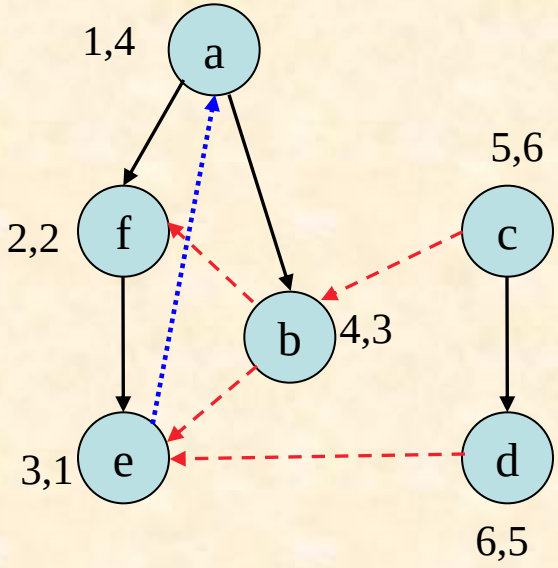


树边　前向边　回边　横跨边

Case 1: 从顶点a开始，
依次访问a,b,e,f; c,d



Case 2: 从顶点a开始，
依次访问a,f,e,b; c,d



为何DFS用于无向图时，不存在前向边及横跨边?

(1)

假设存在前向边，则由前向边定义可设w是v的后裔，且探索(v,w), w已被访问。由于G为无向图，故(v,w)即(w,v)，当深度优先搜索 树构建到w时，首先考虑由w出发的边，故(w,v)在(v,w)之前被探索， 则该边被记为回边，而非前向边。

(2)

假设存在横跨边，则存在v，w都不互为祖先，且w在v之前被访问，否则 (v,w)将成树边；由于访问w时，未访问到v，故(w,v)不存在，矛盾。 故无横跨边。

时间复杂度分析

- 假设图有n个顶点，m条边。图使用邻接链表存储。
- $\Theta(m+n)$

输入：无向图或有向图G= (V,E)
输出：深度优先搜索树(森林)中每个顶点的先序号、后序号

```
1. predfn←0; postdfn←0  //
2. for v∈V
3.   visited[v] ← false    // Θ(n)
4. end for
5. for v∈V    //从一个顶点v出发，可能无法遍历全部顶点
6.   if visited[v] = false  then dfs(v)
7. end for

dfs(v)
1.    visited[v] ←true
2.    predfn←predfn+1
3.    for (v,w)∈E  //总共的循环次数 Θ(m)
4.        if visited[w]=false then dfs(w)
5.    end for
6.    postdfn←postdfn+1.
```

- 如果使用邻接矩阵，时间复杂度？

$$\Theta(m) \longrightarrow \Theta(n^2)$$

深度优先搜索的应用

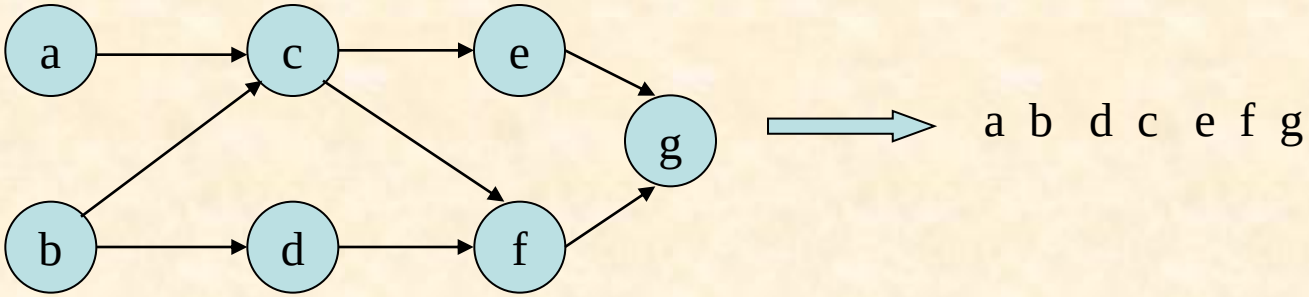
- 图的无回路性判定
- 拓扑排序
- 寻找图的关节点
- 强连通分支
- 网络页面检索

图回路判定

- 问题：若G= (V,E)为一个有n个顶点和m条边的有向或是无向图。要测试G中是否包含有一个回路。
- 方法：对G施加深度优先搜索，如果探测到一个回边，那么可以判定G中含有回路；否则G中无回路。
- 注意：如果G是连通的无向图，则不需要对G进行深度优先搜索来判定是否有回路。G无回路，当且仅当|E|=|V|-1。

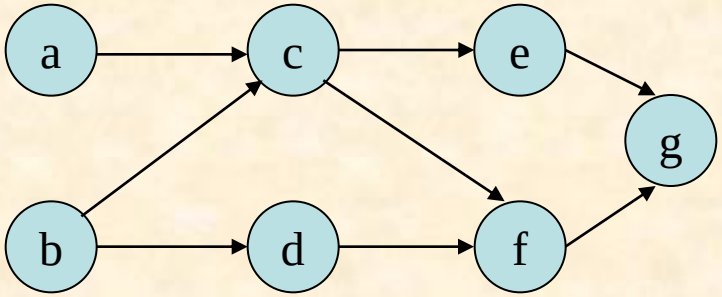
拓扑排序(Topological sorting)

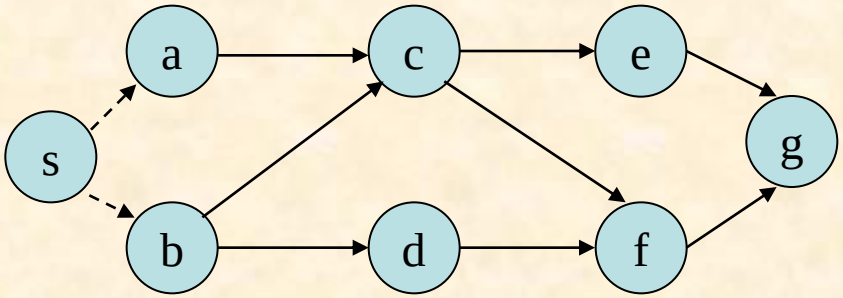
- 给定一个有向无回路图(Directed Acyclic Graph, DAG) G=(V,E)。拓扑排序是为了找到图顶点的一个线性序，使得：如果(v,w) ∈E,那么，在线性序中，v在w之前出现。

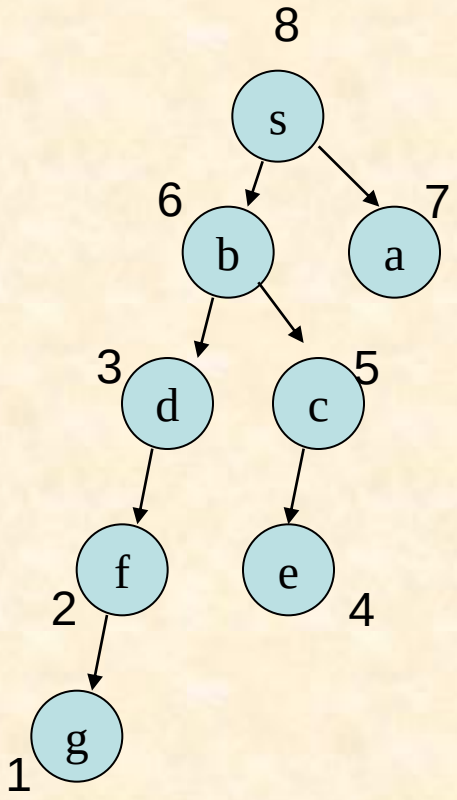


- 我们假设在DAG中只有唯一一个入度为0的顶点；如果有一个以上的顶点入度为0，可以通过添加一个新的顶点s，然后将s指向所有入度为0的顶点，这样s就成为唯一一个入度为0的顶点。

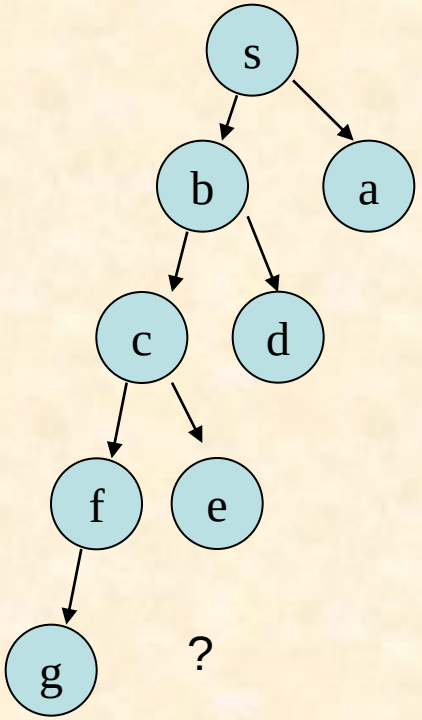
- 拓扑排序的实现：
  - 从入度为0的顶点开始，对DAG实施深度优先搜索。
  - 遍历完成后，计数器postdfn恰好对应于一个在DAG中顶点的反拓扑序。
- 得到拓扑序：
  - 在DFS算法中恰当位置增加输出语句(step 6)，然后将输出结果反转。
  - 在DFS算法中恰当位置增加顶点入栈(step 6)操作，然后依次取栈顶元素输出。







a b c e d f g



寻找关节点(Finding articulation points in a graph*)

- 关节点: 给定无向图G＝(V,E)，|V|>2。称v ∈G 为一关节点，当且仅当存在另外两个不同的顶点u ∈G和w ∈G,并且u与v之间的任意路径均必须经由v通过。
  - 显然，如果G是连通的，那么在移除关节点和与其关联的边后，图变为不连通的。
- 双连通：一个图如果是连通的，并且没有关节点，那么称该图是双连通的。

网络页面检索

- 深度优先搜索是一种在开发爬虫早期使用较多的方法。它的目的是要达到被搜索结构的叶结点(即那些不包含任何超链的HTML文件) 。在一个HTML文件中，当一个超链被选择后，被链接的HTML文件将执行深度优先搜索，即在搜索其余的超链接结果之前必须先完整地搜索单独的一条链。深度优先搜索沿着HTML文件上的超链走到不能再深入为止，然后返回到某一个HTML文件，再继续选择该HTML文件中的其他超链。当不再有其他超链可选择时，说明搜索已经结束。
- 优点是能遍历一个Web 站点或深层嵌套的文档集合；缺点是因为Web结构相当深,，有可能造成一旦进去，再也出不来的情况发生。

广度优先搜索(Breadth-First Search, BFS)

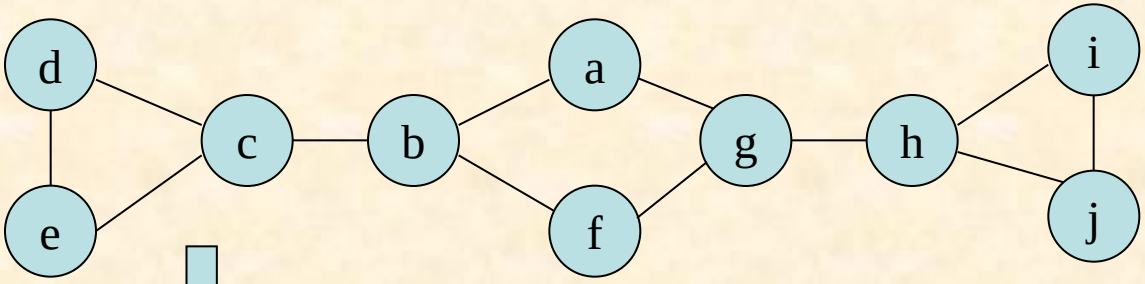
- 思路：在访问一个顶点v后，接下来依次访问邻接于v的所有顶点。对应的搜索树称之为广度优先搜索生成树。
- 实现：使用队列(Queue)
- BFS同样既适用于有向图，亦适用于无向图。
- 在无向图中，边分为：树边或者是横跨边。
- 在有向图中，边分为：树边，回边及横跨边。不存在前向边。

BFS算法
输入：无向图或有向图G= (V,E)
输出：广度优先搜索树中每个顶点的编号(编号：按广度优先的原则，该顶点被访问的次序)

```
1. bfn←0
2. for v∈V
3.   visited[v] ←false
4. end for
5. for v∈V
6.    if visited[v] = false then bfs(v)
7. end for
```

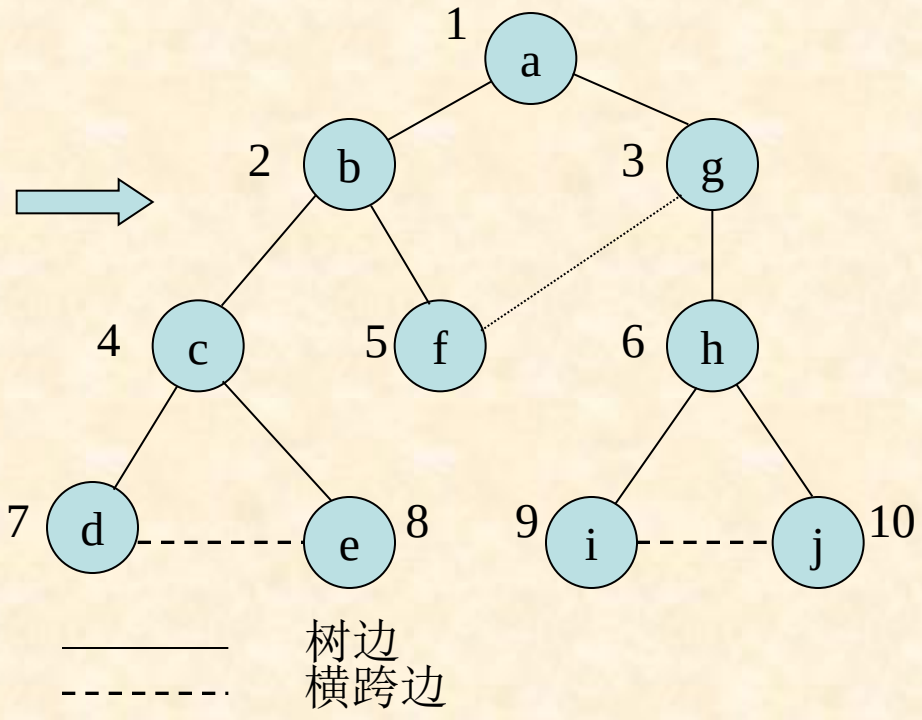
```
bfs(v)
1.  Q←{v}
2.  visited[v] ←true
3.  while Q≠{} //to be visited
4.     v←pop(Q)
5.     bfn←bfn+1
6.     for (v,w)∈E
7.        if visited[w]=false then
8.           Push(w,Q)
9.           visited[w]←true
10.       end if
11.    end for
12. end while
```

无向图BFS实例



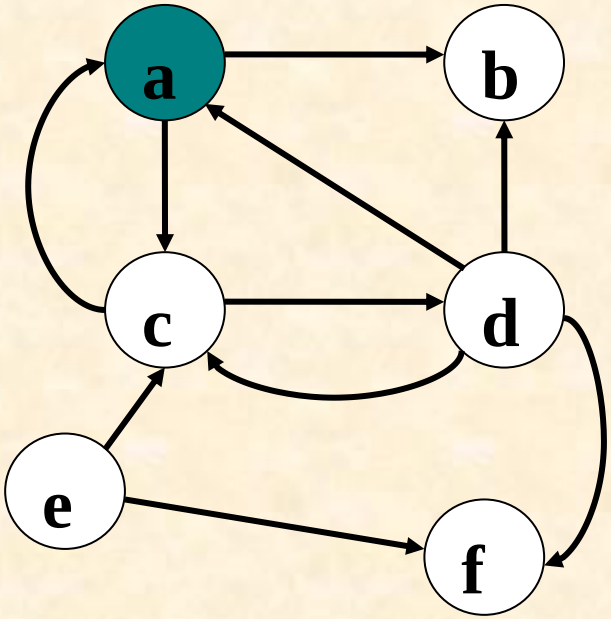
Queue

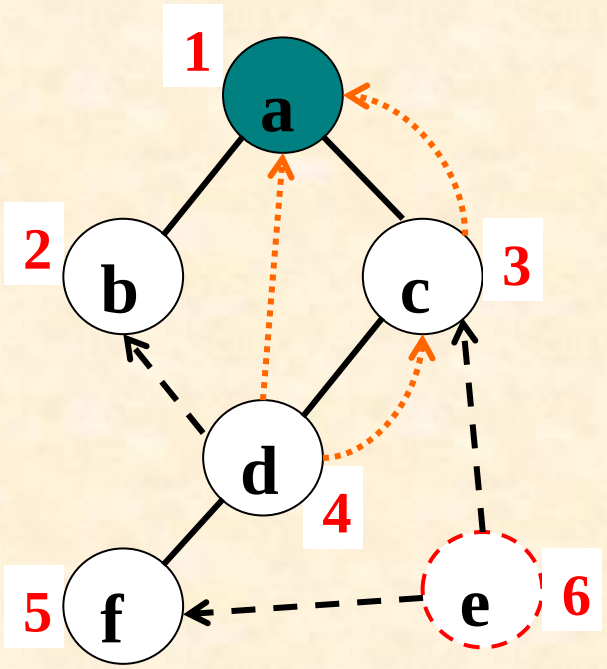
a	b	g	c	f	h	d	e	i	j



时间复杂度： $\Theta(m+n)$

有向图BFS实例





思考：为什么有向图的BFS中不会出现前向边。

1.前向边(Forward edges)—在迄今为止所构建的搜索生成树中，w是v的后裔，并且在探测(v,w)时，w已经被标记为"visited",则(v,w)为前向边。
2.既然要w是v的后裔，那么可以断定，w所在层较v所在的层要低；另一方面，广度优先搜索生成树是逐层产生的，即后裔顶点总是在祖先顶点之后访问。